Contents

基础知识 1
压缩率 1
相对数据冗余 1
冗余的种类 2
自相关 2
信息量 2
信息熵 2
条件熵 3
互信息 3
保真度 4
汉明码 5
压缩技术 8
霍夫曼编码 8
算术编码 10

基础知识

衡量数据压缩的基本单位是比特，也就是二进制位。比特是信息论中的基本单位，它可以表示两种状态，比如 0 和 1，或者是黑和白。比特是信息论中的基本单位，而字节是计算机中的基本单位，一个字节等于 8 个比特。

压缩率

记压缩前的数据大小为 n，压缩后的数据大小为 m，则压缩率为

$$C_R = \frac{n}{m} \in (0, \infty)$$

压缩率越高，说明压缩效果越好。

相对数据冗余

我们希望压缩率一般是一个大于 1 的数，因为这样才能说明压缩后的数据比压缩前的数据更小。那么压缩率的倒数就反映了原始数据中所需的最小携带信息量，换言之，压缩率越大则原始数据中的冗余成分越多。因此，我们可以用压缩率的倒数来衡量数据的冗余程度，常数 1 表示没有冗余，浮点数 0.2 表示有 80% 的冗余。由此，我们定义相对数据冗余为

$$R_D = 1 - \frac{1}{C_R} \in (-\infty, 1)$$

冗余的种类

1. 编码冗余：由于使用了不必要的编码方式，导致数据冗余。最常见的就是使用 8 位二进制表示一个字符，而实际上可能只需要 5 位二进制就可以表示所有的字符了。在解决编码冗余问题时，我们通常对高频字符使用短编码，对低频字符使用长编码，这样可以减少编码冗余。编码冗余是本文所探讨的重点。
2. 像素间冗余 (空间和时间冗余)：在图像中，相邻像素之间的值通常是相似的，因此我们可以通过空间和时间上的冗余来压缩图像。比如，我们可以通过差分编码来压缩视频，因为视频中相邻帧之间的像素值通常是相似的。
3. 心理视觉冗余 (无关信息): 心理视觉冗余可能会导致信息量的丢失。

自相关

自相关用于衡量像素间冗余，计算沿某一方向延伸的像素之间的相关性，自相关的值越大，说明这一方向上的像素值越接近，空间冗余越大。以 y 方向的自相关为例，其定义为

$$\gamma(\Delta n) = \frac{A(\Delta n)}{A(0)} = \frac{\frac{1}{N-\Delta n}\sum_{y=0}^{N-1-\Delta n} f(x,y)f(x,y+\Delta n)}{\frac{1}{N}\sum_{y=0}^{N-1} f^2(x,y)}$$

信息量

如果一件事总是发生或总是不发生，我们认为这件事是没有信息量的，因为它是确定性的。相反，如果一件事发生的概率很小，我们认为这件事的发生会提供更多的信息量。对于一件随机事件 E，规定其概率为 $P(E)$，则事件 E 的信息量为

$$I(E) = \log \frac{1}{P(E)} = -\log P(E)$$

如果 $P(E) = 1$，则 $I(E) = 0$，$P(E)$ 越小，$I(E)$ 越大。

信息熵

信息熵是对信息量的期望，它是对信息量的平均值。对于一个随机事件 X，可能取值的集合为 $\{x_1, x_2, \cdots, x_n\}$，其概率分布为 $P(x_1), P(x_2), \cdots, P(x_n)$，则随机变量 X 的信息熵为

$$H(X) = E[I(X)] = \sum_{i=1}^{n} P(x_i)I(x_i) = -\sum_{i=1}^{n} P(x_i)\log P(x_i)$$

条件熵

条件概率下的信息熵公式可以重写为:

$$H(X|y_i) = -\sum_{j=1}^{n} P(x_j|y_i) \log P(x_j|y_i)$$

全概率公式:

$$P(A) = \sum_{i=1}^{n} P(A|B_i)P(B_i) = \sum_{i=1}^{n} P(A, B_i)$$

给定某一伴随变量 Y，则随机变量 X 的条件熵为

$$\begin{aligned} H(X|Y) &= \sum_{i=1}^{m} P(y_i)H(X|y_i) \\ &= -\sum_{i=1}^{m}\sum_{j=1}^{n} P(y_i)P(x_j|y_i) \log P(x_j|y_i) \\ &= -\sum_{i=1}^{m}\sum_{j=1}^{n} P(x_j, y_i) \log P(x_j|y_i) \end{aligned}$$

互信息

互信息 (也称信息增益) 是目标变量的信息熵与考虑特征变量的条件熵之差，即

$$
\begin{aligned}
I(X;Y) &= H(X) - H(X|Y) \\
&= -\sum_{j=1}^{n} P(x_j)\log P(x_j) + \sum_{i=1}^{m}\sum_{j=1}^{n} P(x_j,y_i)\log P(x_j|y_i) \\
&= -\sum_{j=1}^{n}[\sum_{i=1}^{m} P(x_j,y_i)]\log P(x_j) + \sum_{i=1}^{m}\sum_{j=1}^{n} P(x_j,y_i)\log P(x_j|y_i) \\
&= \sum_{i=1}^{m}\sum_{j=1}^{n} P(x_j,y_i)[\log P(x_j|y_i) - \log P(x_j)] \\
&= \sum_{i=1}^{m}\sum_{j=1}^{n} P(x_j,y_i)\log\frac{P(x_j|y_i)}{P(x_j)} \\
\text{分式上下同乘}P(y_i) &= \sum_{i=1}^{m}\sum_{j=1}^{n} P(x_j,y_i)\log\frac{P(x_j,y_i)}{P(x_j)P(y_i)} \\
\text{或分母扩展为全概公式} &= \sum_{i=1}^{m}\sum_{j=1}^{n} P(x_j,y_i)\log\frac{P(x_j|y_i)}{\sum_{k=1}^{m} P(x_j,y_k)}
\end{aligned}
$$

互信息的性质包括：

1. 对称性：$I(X;Y) = I(Y;X)$；从上式可以看出 $I(X;Y) = \sum_{i=1}^{m}\sum_{j=1}^{n} P(x_j,y_i)\log\frac{P(x_j,y_i)}{P(x_j)P(y_i)}$，交换 X 和 Y 的位置依然成立，所以互信息是对称的。
2. 当 X 和 Y 相互独立时，$P(x_j,y_i) = P(x_j)P(y_i)$，此时 $I(X;Y) = 0$，即 X 和 Y 相互独立时，信息熵和条件熵相等，信息增益为 0。
3. 可加性：$I(X;Y,Z) = I(X;Y) + I(X;Z|Y)$；联合事件 Y 和 Z 所提供的关于 X 的信息量等于 Y 所提供的关于 X 的信息量加上 Z 所提供的关于 X 的信息量在给定 Y 的条件下的条件熵。

保真度

令 $\hat{f}(x,y)$ 表示压缩解压后的图像，$f(x,y)$ 表示原始图像，则压缩误差为

$$
e(x,y) = f(x,y) - \hat{f}(x,y)
$$

总误差为 $\sum_{x=0}^{M-1}\sum_{y=0}^{N-1} e(x,y)$，我们一般使用均方误差来衡量压缩的保真度，即

$$
MSE = \frac{1}{MN}\sum_{x=0}^{M-1}\sum_{y=0}^{N-1} e^2(x,y)
$$

或使用均方根误差

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{MN}\sum_{x=0}^{M-1}\sum_{y=0}^{N-1}[f(x,y)-\hat{f}(x,y)]^2}$$

均方信噪比 (SNR_{ms}) 是信号/解压图像的功率与噪音/误差的功率之比，即

$$\text{SNR}_{ms} = \frac{P_{signal}}{P_{noise}} = \frac{\sum_{x=0}^{M-1}\sum_{y=0}^{N-1}\hat{f}^2(x,y)}{\sum_{x=0}^{M-1}\sum_{y=0}^{N-1}e^2(x,y)}$$

汉明码

汉明码是一种纠错码，在传输数据中插入冗余的校验信息，使得数据内部实现多组校验。假设我们要传输的实际有效数据长度为 n，校验位个数为 k，那么应当满足：

$$2^k \geq n + k + 1$$

```python
def get_k(n):
    k = 0
    while 2 ** k < n + k + 1:
        k += 1
    return k
```

得到 k 的大小后，我们就得到了待校验位的编码总长 $n+k$，其中的校验位为 $\{1, 2, 4, \dots, 2^{k-1}\}$，其余位为待校验位。这里我们要引入校验组的概念：对于第 i 个校验位，易得其二进制索引为 $1\underbrace{0\cdots0}_{i-1}$，那么该校验位 i 对应的校验组为所有二进制表示中倒数第 i 位为 1 的数据位，如 4 的二进制表示为 100，那么 4 对应的校验组为 $\{5(101), 6(110), 7(111), 12(1100), \cdots\}$。

$$G_{2^{i-1}} = \{j | j\text{的二进制形如} \dots \text{xxx}1\underbrace{\text{xxx}}_{i-1}\}$$

```python
def get_check_groups(n, k):
    check_groups = {}
    for i in range(1, k + 1):
        index = 2 ** (i - 1)
        check_groups[index] = []
        for j in range(index + 1, n + k + 1):
            if j & (1 << (i - 1)):
                check_groups[index].append(j)
    return check_groups
```

给定校验位和校验组的概念后，我们就可以计算校验位的值了。首先我们在非校验位填入待传输的数据，然后从小到大依次计算每个校验位的值，校验位的值取决于校验组采用的是奇校验还是偶校验，对于奇校验，校验位的值为校验组中

所有数据位的异或值，对于偶校验，校验位的值为校验组中所有数据位的或值。

$$b_{2^{i-1}} = \oplus_{j \in G_{2^{i-1}} \wedge j \neq 2^{i-1}} b_j$$

检验错误位时，我们从小到大遍历校验位，对于每个校验位，我们取出其对应的校验组，然后计算校验组中所有数据位的异或值（包括该校验位），如果异或值为 1，则说明该校验位对应的数据位有错，否则该校验位对应的数据位无错，记录该异或值为 c_i。最后得到的二进制表示 $c_k c_{k-1} \cdots c_1$ 就是错误位的索引，如果 $c_k c_{k-1} \cdots c_1 = 0$，则说明没有错误。

$$c_{2^{i-1}} = \oplus_{j \in G_{2^{i-1}} \cup \{2^{i-1}\}} b_j$$

```python
def get_check_code(group_data, parity):
    """
    group_data: 校验组中的数据位
    parity: 奇校验或偶校验
    :return: 校验位的值
    """
    if parity == 'odd':
        return 1 - sum(group_data) % 2
    elif parity == 'even':
        return sum(group_data) % 2
    else:
        raise ValueError('parity must be odd or even')

class HammingCode:
    def __init__(self, data: (list, tuple, str, int)=None):
        if data is not None:
            self._process(data)

    def _process(self, data: (list, tuple, str, int)):
        if isinstance(data, str):
            data = [int(i) for i in data]
        elif isinstance(data, int):
            # 转为二进制列表
            data = [int(i) for i in bin(data)[2:]]
        self.data = data
        self.n = len(data)
        self.k = get_k(self.n)
        self.check_groups = get_check_groups(self.n, self.k)

    def _inverse_print(self, code):
        print("Hamming Code: {}".format("".join([str(i) for i in code[::-1]])))

    def _check_error(self, code=None):
        if code is None:
            code = self.code
```

```python
        errors = []
        for i, group in self.check_groups.items():
            res = get_check_code([code[j - 1] for j in group + [i]], 'odd')
            if res: # 有错
                errors.append(1) # 补1
            else:
                errors.append(0) # 补0
        if sum(errors) == 0:
            return None
        else:
            return sum([2 ** i for i, e in enumerate(errors) if e])

        if len(errors) == 0:
            return None
        else:
            return errors

    def __call__(self, data=None):
        if data is not None:
            self._process(data)
        return self.encode()

    def encode(self):
        # 默认奇校验
        code = [0] * (self.n + self.k)
        data_index = 0
        for i in range(1, self.n + self.k + 1):
            if i in self.check_groups:
                code[i - 1] = None
            else:
                code[i - 1] = self.data[self.n - data_index - 1]
                data_index += 1
        for i, group in self.check_groups.items():
            code[i - 1] = get_check_code([code[j - 1] for j in group], 'odd')
        # 倒序输出字符串
        self._inverse_print(code)
        self.code = code
        return code

    def decode(self, code=None):
        if code is None:
            code = self.code
        # 默认奇校验
        errors = self._check_error(code)
        if errors is not None:
            code[errors - 1] = 1 - code[errors - 1]
        data = []
        for i in range(1, self.n + self.k + 1):
            if i not in self.check_groups:
```

```
                data.append(code[i - 1])
        self.data = data[::-1]
        return "".join([str(i) for i in data[::-1]])
```

压缩技术

主要是针对编码冗余的压缩技术，即如何用更少的比特来表示一个字符。我们定义编码单位对象 (一般是字母) 为 $\{a_1, a_2, \cdots, a_n\}$，其概率分布为 $P(a_1), P(a_2), \cdots, P(a_n)$，对应的编码长度 (bit) 为 $l_1, l_2, \cdots, l_n$，则平均编码长度为

$$l_{avg} = \sum_{i=1}^{n} P(a_i) l_i$$

如果所有的编码长度都相同，那么 $l_{avg} = l$；我们的目的是使 l_{avg} 尽可能小，即使得编码冗余尽可能小，一种合理的思路是让高频字符的编码更短，即 $P(a_i)$ 越大，l_i 越小，并且保证编码的唯一性。前文提到信息量 $I = -\log P$，因此我们可以说信息量越小，编码长度越短，这也是合理的，所以我们令编码长度 (整数) 与信息量相关联，即

$$\log \frac{1}{P(a_i)} \leq l_i < \log \frac{1}{P(a_i)} + 1$$

代入平均编码长度公式，得到

$$\sum_{i=1}^{n} P(a_i) \log \frac{1}{P(a_i)} \leq \sum_{i=1}^{n} P(a_i) l_i < \sum_{i=1}^{n} P(a_i) \log \frac{1}{P(a_i)} + \sum_{i=1}^{n} P(a_i)$$

即

$$H(X) \leq l_{avg} < H(X) + 1$$

霍夫曼编码

霍夫曼编码的基本思想是：将出现频率较高的字符用较短的编码，出现频率较低的字符用较长的编码，并自底向上依次合并低频信源，直至合并到根节点；在合并的过程中，左子树编码为 0，右子树编码为 1。霍夫曼编码是一种前缀编码，即任何一个字符的编码都不是另一个字符编码的前缀，这样可以保证解码的唯一性。

霍夫曼编码的过程如下：

1. 将信源按照概率从小到大排序
2. 将概率最小的两个信源合并，得到一个新的信源，其概率为两个信源概率之和
3. 重复步骤 2，直至合并到根节点

4. 从根节点开始，左子树编码为 0，右子树编码为 1
5. 重复步骤 4，直至叶子节点
6. 得到每个信源的编码

```python
# 信源
class Node:
    def __init__(self, symbol, prob):
        self.symbol = symbol
        self.prob = prob
        self.left = None
        self.right = None
        self.code = None

# 信源按照概率从小到大排序
def sort_nodes(nodes):
    return sorted(nodes, key=lambda x: x.prob)

# 合并两个信源
def merge_nodes(node1, node2):
    node = Node(None, node1.prob + node2.prob)
    node.left = node1
    node.right = node2
    return node

# 生成霍夫曼编码
def huffman(nodes):
    # 信源按照概率从小到大排序
    nodes = sort_nodes(nodes)
    # 重复合并两个信源，直至合并到根节点
    while len(nodes) > 1:
        node1 = nodes.pop(0)
        node2 = nodes.pop(0)
        node = merge_nodes(node1, node2)
        for i in range(len(nodes)):
            if nodes[i].prob > node.prob:
                nodes.insert(i, node)
                break
        else:
            nodes.append(node)
    # 从根节点开始，左子树编码为0，右子树编码为1
    codes = {}
    root = nodes[0]
    root.code = ''
    stack = [root]
    while stack:
        node = stack.pop()
        if node.symbol:
            codes[node.symbol] = node.code
        if node.left:
            node.left.code = node.code + '0'
```

```
            stack.append(node.left)
        if node.right:
            node.right.code = node.code + '1'
            stack.append(node.right)
    # 输出编码
    return codes
```

```
probs = [0.1, 0.2, 0.3, 0.4]
symbols = ['A', 'B', 'C', 'D']
nodes = [Node(symbol, prob) for symbol, prob in zip(symbols, probs)]
codes = huffman(nodes)
print(codes)
```

```
{'B': '111', 'A': '110', 'C': '10', 'D': '0'}
```

算术编码

算术编码的基本思想是：将整个信源看作一个整体，将整个信源的编码看作一个区间，区间的左端点为 0，右端点为 1，区间的长度为 1，每个信源的编码都是区间的一个子区间，子区间的长度等于信源的概率。算术编码是一种无前缀编码，即任何一个字符的编码都不是另一个字符编码的前缀，这样可以保证解码的唯一性。

```
from collections import Counter

string = "ABAACCAACB"

def float2bin(f):
    if f >= 1 or f <= 0:
        return "Error"
    res = "0."
    while f > 0:
        f *= 2
        if f >= 1:
            res += "1"
            f -= 1
        else:
            res += "0"
    return res

def arithmetic_encode(string):

    probs = Counter(string)
    probs = {k: v / len(string) for k, v in probs.items()}
    left = 0
    intervals = {}
    for k in probs:
        intervals[k] = (left, left + probs[k])
        left += probs[k]

```

```
    left, right = 0, 1
    for c in string:
        l, r = intervals[c] # 信源的编码区间
        interval = right - left # 区间长度
        right = left + interval * r # 更新右端点
        left = left + interval * l # 更新左端点
        # print(c, (left, right))
    midpoint = (left + right) / 2 # 区间的中点
    return float2bin(midpoint)[2:] # 转为二进制，去掉前面的0.
```